

Smart Bank API Testing Guide

Prerequisites

- Postman installed
- Application running on `http://localhost:8080`
- Admin user created (automatic on startup)

Test Flow (Recommended Order)

1. Authentication Flow

Test 1.1: Register New Customer

POST /api/auth/register

Body:

```
{  
  "email": "john@example.com",  
  "password": "John@123",  
  "fullName": "John Doe",  
  "phone": "9876543210"  
}
```

Expected: 201 CREATED

Check console for OTP code

Test 1.2: Verify OTP

POST /api/auth/verify-otp

Body:

```
{  
  "email": "john@example.com",  
  "otp": "<OTP_FROM_CONSOLE>"  
}
```

Expected: 200 OK

Test 1.3: Login as Customer

POST /api/auth/login

Body:

```
{  
  "email": "john@example.com",  
  "password": "John@123"  
}
```

Expected: 200 OK with JWT token

SAVE THE TOKEN for next requests!

Test 1.4: Login as Admin

POST /api/auth/login

Body:

```
{  
  "email": "admin@bank.com",  
  "password": "Admin@123"  
}
```

Expected: 200 OK with JWT token

SAVE THIS TOKEN separately!

2. Account Management

Test 2.1: Create Savings Account

POST /api/accounts

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Body:

```
{  
  "type": "SAVINGS"  
}
```

Expected: 201 CREATED

Note the accountNumber!

Test 2.2: Create Current Account

POST /api/accounts

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Body:

```
{  
  "type": "CURRENT"  
}
```

Expected: 201 CREATED

Test 2.3: Get All My Accounts

GET /api/accounts

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 200 OK with list of accounts

3. Fund Transfer

Test 3.1: Create Second User (for transfer testing)

Repeat registration/verification for another user to have a receiver account.

Test 3.2: Transfer Funds

POST /api/transfer

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Body:

```
{  
  "fromAccountNumber": "<YOUR_ACCOUNT>",  
  "toAccountNumber": "<OTHER_USER_ACCOUNT>",  
  "amount": 1000.00,  
  "description": "Test transfer"  
}
```

Expected: 200 OK

Test 3.3: Verify Transfer Failed (Insufficient Balance)

POST /api/transfer

Body:

```
{  
  "amount": 999999.00  
}
```

Expected: 400 BAD REQUEST

Error: "Insufficient balance"

4. Transaction History

Test 4.1: Get Account Transactions

GET /api/transactions/<ACCOUNT_NUMBER>

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 200 OK with transaction list

Test 4.2: Get Recent Transactions

GET /api/transactions/<ACCOUNT_NUMBER>/recent

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 200 OK with last 10 transactions

5. Loan Management

Test 5.1: Apply for Loan

POST /api/loans

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Body:

```
{  
  "amount": 50000.00,  
  "tenureMonths": 12,  
  "purpose": "Home renovation"  
}
```

Expected: 201 CREATED

Note the loanId!

Test 5.2: Try to Apply Again (Should Fail)

POST /api/loans
Body: (same as above)
Expected: 400 BAD REQUEST
Error: "You already have an active loan"

Test 5.3: Get My Loans

GET /api/loans
Headers:
Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 200 OK with list of loans

Test 5.4: Get Loan Details

GET /api/loans/<LOAN_ID>
Headers:
Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 200 OK with loan details

6. Admin Operations

Test 6.1: Get Pending Loans (Admin Only)

GET /api/admin/loans/pending
Headers:
Authorization: Bearer <ADMIN_TOKEN>

Expected: 200 OK with pending loans

Test 6.2: Try with Customer Token (Should Fail)

GET /api/admin/loans/pending
Headers:
Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 403 FORBIDDEN

Test 6.3: Approve Loan

PUT /api/admin/loans/<LOAN_ID>/approve

Headers:

Authorization: Bearer <ADMIN_TOKEN>

Body:

```
{  
  "approved": true,  
  "remarks": "Approved based on credit score"  
}
```

Expected: 200 OK

Test 6.4: Mark Loan Under Review

PUT /api/admin/loans/<LOAN_ID>/review

Headers:

Authorization: Bearer <ADMIN_TOKEN>

Expected: 200 OK

Test 6.5: Reject Loan

PUT /api/admin/loans/<LOAN_ID>/approve

Headers:

Authorization: Bearer <ADMIN_TOKEN>

Body:

```
{  
  "approved": false,  
  "remarks": "Insufficient credit score"  
}
```

Expected: 200 OK

Test 6.6: Get Loans by Status

GET /api/admin/loans?status=APPROVED

Headers:

Authorization: Bearer <ADMIN_TOKEN>

Expected: 200 OK with approved loans

7. Validation Tests

Test 7.1: Invalid Email Format

POST /api/auth/register

Body:

```
{  
  "email": "invalid-email",  
  "password": "Test@123",  
  "fullName": "Test",  
  "phone": "1234567890"  
}
```

Expected: 400 BAD REQUEST

Errors: "Invalid email format"

Test 7.2: Weak Password

POST /api/auth/register

Body:

```
{  
  "email": "test@test.com",  
  "password": "weak",  
  "fullName": "Test",  
  "phone": "1234567890"  
}
```

Expected: 400 BAD REQUEST

Errors: "Password must be 8-50 characters", "Password must contain..."

Test 7.3: Invalid Phone Number

POST /api/auth/register

Body:

```
{  
  "email": "test@test.com",  
  "password": "Test@123",  
  "fullName": "Test",  
  "phone": "123"  
}
```

Expected: 400 BAD REQUEST

Error: "Phone must be 10 digits"

Test 7.4: Duplicate Email

POST /api/auth/register

Body:

```
{  
  "email": "john@example.com",  
  "password": "Test@123",  
  "fullName": "Test",  
  "phone": "1234567890"  
}
```

Expected: 409 CONFLICT

Error: "Email 'john@example.com' is already registered"

Test 7.5: Loan Amount Too Low

POST /api/loans

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Body:

```
{  
  "amount": 5000.00,  
  "tenureMonths": 12,  
  "purpose": "Test"  
}
```

Expected: 400 BAD REQUEST

Error: "Minimum loan amount is ₹10,000.0"

Test 7.6: Loan Amount Too High

POST /api/loans

Body:

```
{  
  "amount": 99999999.00,  
  "tenureMonths": 12,  
  "purpose": "Test"  
}
```

Expected: 400 BAD REQUEST

Error: "Maximum loan amount is ₹10,000,000.0"

8. Security Tests

Test 8.1: Access Protected Endpoint Without Token

GET /api/accounts

Expected: 401 UNAUTHORIZED or 403 FORBIDDEN

Test 8.2: Access with Invalid Token

GET /api/accounts

Headers:

Authorization: Bearer invalid_token_here

Expected: 401 UNAUTHORIZED

Test 8.3: Access with Expired Token

(Test after 24 hours, or modify JWT_EXPIRATION_MS for quick test)

Expected: 401 UNAUTHORIZED

9. Edge Cases

Test 9.1: Transfer to Same Account

POST /api/transfer

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Body:

```
{  
  "fromAccountNumber": "ACC123",  
  "toAccountNumber": "ACC123",  
  "amount": 100.00  
}
```

Expected: Should work or return meaningful error

Test 9.2: Transfer with Negative Amount

POST /api/transfer

Body:

```
{  
  "fromAccountNumber": "ACC123",  
  "toAccountNumber": "ACC456",  
  "amount": -100.00  
}
```

Expected: 400 BAD REQUEST

Error: "Amount must be positive"

Test 9.3: Access Another User's Account

GET /api/accounts/<OTHER_USER_ACCOUNT_NUMBER>

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 403 FORBIDDEN

Error: "You don't have access to this account"

Test 9.4: Access Another User's Loan

GET /api/loans/<OTHER_USER_LOAN_ID>

Headers:

Authorization: Bearer <CUSTOMER_TOKEN>

Expected: 403 FORBIDDEN (if implemented) or returns loan but shouldn't

Expected Results Summary

Test Category	Total Tests	Expected Pass
Authentication	4	4
Account Management	3	3
Fund Transfer	3	3
Transactions	2	2
Loans	4	4
Admin Operations	6	6
Validation	6	6

Test Category	Total Tests	Expected Pass
Security	3	3
Edge Cases	4	4
TOTAL	35	35

Troubleshooting

Issue: "No authenticated user found"

Solution: Make sure you included the JWT token in Authorization header

Issue: "403 Forbidden" on admin endpoint

Solution: Make sure you're using the admin token, not customer token

Issue: "Insufficient balance"

Solution: Accounts start with 0 balance. You need to deposit first (or manually update DB for testing)

Issue: OTP not showing in console

Solution: Check application logs, OTP is printed to console when `show-sql: true`

Issue: "Loan has already been processed"

Solution: You're trying to approve/reject a loan that's already approved/rejected

Database Verification

After each test, you can verify in database:

```
sql
```

-- Check users

```
SELECT id, email, full_name, role, is_verified FROM users;
```

-- Check accounts

```
SELECT id, account_number, balance, type, status, user_id FROM accounts;
```

-- Check transactions

```
SELECT id, type, amount, balance_after, transaction_ref, created_at  
FROM transactions  
ORDER BY created_at DESC  
LIMIT 10;
```

-- Check loans

```
SELECT id, loan_number, amount, status, applied_date, approved_date  
FROM loans  
ORDER BY applied_date DESC;
```

Next Steps

After all tests pass:

1. Deploy to production environment
2. Configure production database
3. Set environment variables for secrets
4. Enable HTTPS
5. Set up monitoring and logging
6. Create backup strategy